

БАЗА ЗНАНИЙ

Как поднять свою LLM и подключить к Codex

Как поднять локальную LLM через Ollama и Open WebUI и подключить её к Codex CLI: рабочий контур для приватной работы с моделями без утечки данных.

ОПУБЛИКОВАНО 15.07.2026

ИИ-агенты

DevTools

Backend

КРАТКИЙ АУДИООБЗОР

Слушать материал

Альфа-версия: аудио подготовлено через Yandex SpeechKit, возможны ошибки.

0:00 / 2:19

0.85x

1x

1.25x

1.5x

2.0x

2 мин 20 сек

Скачать MP3

Практический контур: как поднять локальную LLM через [Ollama](#) и [Open WebUI](#) и подключить её к Codex CLI, чтобы код и данные не уходили в облако. Это не обзор одного инструмента, а сборка рабочего пайплайна от установки модели до её использования в кодинг-агенте.



Главная мысль: локальная модель не заменит фронтит на сложном коде, но закрывает то, чего облако не даёт: работу с приватными данными на своём железе. Разумная схема гибридная — фронтит для тяжёлых задач, локальная LLM для чувствительного и чернового.

Что решает этот контур

Облачные модели удобны, но каждый запрос уходит на чужие серверы. Для части задач это неприемлемо: персональные данные клиентов, внутренние документы, коммерческая тайна, код под NDA.

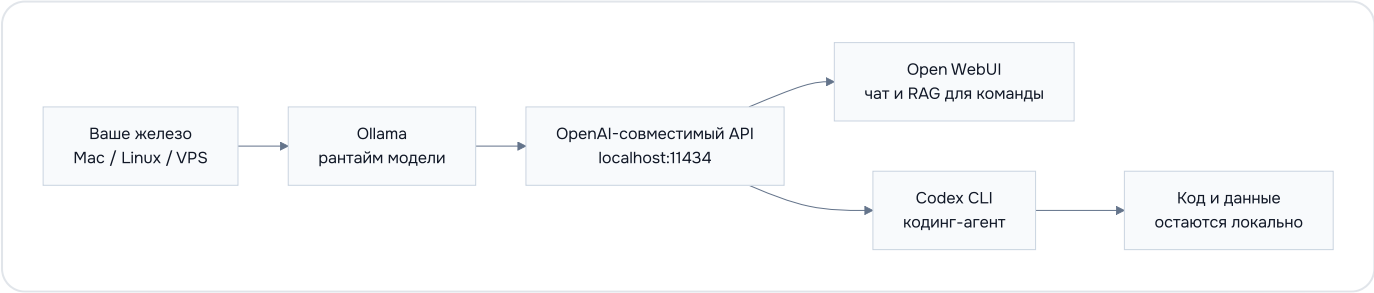
Локальный контур закрывает три проблемы:

- **Приватность.** Промпты и файлы не покидают вашу машину.
- **Стоимость на объёме.** Нет оплаты за токены: платите один раз за железо.
- **Автономность.** Работает без интернета и не зависит от лимитов и блокировок провайдера.

Взамен вы берёте на себя железо, обновления моделей и то, что локальная модель слабее топовой облачной.

Как выглядит контур

Три звена: рантайм модели, интерфейс для людей и коддинг-агент. Все обращаются к одной локальной модели по единому API.



Почему именно так: Ollama поднимает модель и отдаёт её по OpenAI-совместимому протоколу. За счёт этого и Open WebUI, и Codex CLI подключаются к ней теми же настройками, что и к облачному OpenAI: меняется только адрес.

Прerequisites

До старта проверьте, что готово:

- **Железо под модель.** Главное ограничение — память. Ориентиры в таблице ниже.
- **Docker** — для Open WebUI, ставится одной командой.
- **Codex CLI** — установлен и авторизован.
- **Выбранная модель** — под коддинг берите модель с поддержкой tool calling (например, семейство Qwen Coder).

Размер модели	Нужно памяти (RAM/VRAM)	Подходит для
7–8B	~8 ГБ	ноутбук, быстрые черновики
14B	~16 ГБ	баланс качества и скорости
32B	~24–32 ГБ	серьёзный коддинг

70B+

48 ГБ и выше

максимум качества,
нужна станция



Внимание: неставляйте Ollama напрямую в интернет. По умолчанию он слушает только localhost — так и оставьте. Для доступа с других устройств используйте VPN или закрытый туннель, иначе к вашей модели получит доступ кто угодно.

Шаг 1. Поднять модель в Ollama

Установите рантайм Ollama и скачайте модель:

```
# macOS и Linux
curl -fsSL https://ollama.com/install.sh | sh

# скачать кодированную модель (пример)
ollama pull qwen2.5-coder:14b
```

На macOS есть и второй путь — приложение с сайта (.dmg); оба варианта ставят одну и ту же CLI ollama.

Ollama запускается как фоновый сервис и слушает порт 11434. Проверьте, что модель отвечает по OpenAI-совместимому API:

```
curl http://localhost:11434/v1/models
```

Если в ответе есть ваша модель — рантайм готов.

Шаг 2. Поднять Open WebUI

Open WebUI даёт человеческий интерфейс к локальной модели: чат, историю, доступ для команды и RAG по вашим документам. Запуск через Docker:

```
docker run -d -p 3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data \
  --name open-webui \
  ghcr.io/open-webui/open-webui:main
```

Откройте `http://localhost:3000` и создайте локальную учётную запись администратора. Open WebUI сам находит Ollama по адресу `host.docker.internal:11434`. Если модель не подхватилась, пропишите адрес вручную в настройках подключений.



Совет: RAG в Open WebUI работает на той же локальной модели, поэтому документы, которые вы загружаете для поиска, тоже не уходят в облако. Это удобный способ дать команде приватный чат по внутренней базе.

Шаг 3. Подключить Codex CLI к локальной модели

Codex CLI умеет работать с любым OpenAI-совместимым провайдером. Сначала опишите локального провайдера в `~/.codex/config.toml`:

```
# Локальный провайдер поверх Ollama
[model_providers.ollama]
name = "Ollama (local)"
```

```
base_url = "http://localhost:11434/v1"
# Ollama не проверяет ключ, но поле может быть обязательным — подойдёт любая
env_key = "OLLAMA_API_KEY"
```

Сам профиль в свежих версиях Codex CLI описывается отдельным файлом рядом с конфигом — например `~/.codex/local.config.toml`. Ключи кладутся на верхний уровень, без вложенной таблицы:

```
model = "qwen2.5-coder:14b"
model_provider = "ollama"
```

 **Важно про версии:** раньше профиль писали таблицей `[profiles.local]` прямо в `config.toml`. В свежих версиях Codex CLI такой формат не читается, а устаревший блок иногда мешает запуску — профиль должен лежать отдельным файлом `<имя>.config.toml`. Точные имена полей сверяйте с актуальной документацией, они меняются между версиями.

Запустите Codex на локальном профиле:

```
export OLLAMA_API_KEY="local"
codex --profile local
```

Если вам хватает открытых моделей `gpt-oss`, есть путь короче: флаг `--oss` сам поднимает связку с Ollama без ручной правки конфига.

```
codex --oss -m gpt-oss:20b
```

Тот же контур в Codex App и IDE

CLI — не единственный клиент Codex. Desktopное приложение и расширение для редактора читают тот же конфиг `~/.codex/config.toml`, поэтому подключаются к локальной модели теми же настройками.

- **Codex App (macOS/Windows).** Проще всего поднять через Ollama — команда сама пропишет локальный эндпоинт и откроет приложение:

```
ollama launch codex-app
# с конкретной моделью
ollama launch codex-app --model qwen2.5-coder:14b
```

Поддержка Codex App появилась в Ollama версии 0.24.0 и новее.

- **IDE-расширение (VS Code / Codium).** Работает на том же `config.toml`: откройте настройки Codex → «Open config.toml» и укажите локального провайдера. Переключение модели пока только через правку конфига с перезапуском редактора — штатного селектора для локальных моделей нет.

 **Нюанс:** при переходе на локальную модель Codex App меняет интерфейс — вместо облачного селектора появляется переключатель reasoning-усилия. Обратный откат на облачные модели иногда подвисает, поэтому переключайтесь осознанно.

Шаг 4. Проверить, что всё работает

- ☐ `curl http://localhost:11434/v1/models` возвращает вашу модель
- ☐ Open WebUI открывается и отвечает в чате
- ☐ Codex на профиле `local` выполняет простую задачу (например, объясняет файл)
- ☐ Во время работы Codex нет исходящих запросов к `api.openai.com` (проверьте в мониторе сети)

Последний пункт — главный: он подтверждает, что данные действительно остаются локально.

Сценарии использования

Куда локальный контур встаёт лучше всего:

- **Код под NDA и приватные репозитории.** Ревью, рефакторинг и объяснение легаси-кода, который нельзя отправлять в облако. Codex на профиле `local` читает и правит файлы прямо в рабочей папке.
- **Чувствительные данные.** Логи с персональными данными, выгрузки из БД, внутренние отчёты: попросите Codex написать и прогнать скрипт обработки — сами данные не покидают машину.
- **Приватный чат по внутренней базе.** Open WebUI с RAG превращает локальную модель в справочную по вашим документам для всей команды, без утечки содержимого.
- **Работа офлайн и в изолированном контуре.** Самолёт, объект без интернета, air-gapped-сеть — модель уже на вашем железе и не зависит от связи.
- **Рутина на объёме.** Генерация тестов, докстрингов, черновых коммит-сообщений и однотипных правок, где не хочется платить за токены и упираться в лимиты.

- **Гибридный день.** Чувствительное и рутинное — на локальной модели, сложную архитектуру — переключением на облачный Codex. Один инструмент, разные профили.



Пример: нужно разобрать дамп логов с персональными данными.

Запускаете `codex --profile local` в папке с файлом и просите: «прочитай access.log, найди топ-20 IP по числу ошибок 5xx и собери CSV». Модель читает файл, пишет скрипт и отдаёт результат — лог никуда не уходит.

Типичные ошибки

- **Модель не влезает в память.** Система уходит в своп, ответы идут минутами. Возьмите модель на размер меньше или квантованную версию.
- **Codex молчит или падает на инструментах.** Кодинг-агенту нужна модель с поддержкой tool calling. Обычные чат-модели не подойдут.
- **Open WebUI не видит Ollama.** Чаще всего дело в сети Docker: проверьте флаг `--add-host=host.docker.internal:host-gateway` и адрес подключения.
- **Порт 11434 недоступен снаружи.** Так и должно быть: доступ закрыт по умолчанию. Открывайте его только через VPN или туннель.

Когда это оправдано



Компромисс: локальная LLM выигрывает в приватности и стоимости на объёме, но проигрывает фронтину в качестве сложного кода и скорости на слабом железе. Держите гибрид: чувствительные и рутинные задачи на локальной модели, тяжёлую разработку на облачном Codex.

Антипаттерны

- ❌ Ставить 70В на ноутбук и ждать нормальной скорости — упрётесь в память
- ❌ Брать чат-модель для кодинг-агента без поддержки tool calling
- ❌ Выставлять Ollama в интернет ради удалённого доступа вместо VPN
- ❌ Полностью отказываться от фронта — на сложном коде локальная модель проседает
- ❌ Грузить приватные документы в облачный RAG, когда весь смысл контура в локальности

По теме

- Статья: [Почему я не использую Claude Code и сделал ставку на Codex](#)
- Блог: [Почему стоит владеть своим workflow, а не оболочкой от большой лаборатории](#)
- База знаний: [OpenAI Codex — облачный coding-агент для параллельной разработки](#)

для АГЕНТА

Прочитать с ИИ

Короткий промт для резюме, выводов и применения к вашей задаче.

</> Копировать промт

 ChatGPT

 Claude

Если вы работаете с чувствительными данными или просто не хотите зависеть от облака, локальный контур даёт спокойствие: модель, код и документы остаются на вашем железе.

Если захотите обсудить, как это применить у себя или в команде — пишите в Telegram [@pimenov](https://t.me/pimenov)

Продолжить по теме

Быстрый путь дальше:
похожие материалы,
темы, разборы или анонс
программы.

[Найти похожее](#)

[Темы](#)

[Разборы](#)

[Перезагрузка](#)

Если хотите разобрать свою задачу — напишите мне

Можно прийти с идеей, черновым контекстом или уже живой задачей. Помогу быстро понять, где реальный следующий шаг, а где лишний шум.

[Напишите мне в Телеграм](#)

[Мой Телеграм канал](#)

Обычно хватает 2–3 сообщений, чтобы понять, могу ли я здесь реально помочь и в каком формате лучше двигаться дальше.

© 2026 ИП Пименов Сергей Викторович
ИНН 616271176890
ОГРН 316619600255641

[Политика обработки персональных данных](#)

[v1.1.0](#)